Acknowledgement:

We sincerely express our heartfelt gratitude to our project guide **Dr. Vyas Murnal** for his invaluable guidance, constant encouragement, and support throughout the successful completion of this project. We are also thankful to **Dr. Shreedhar A. Joshi (HOD)** and **Dr. Ramesh L. Chakrasali (Principal)** for providing the necessary facilities and academic support that made this work possible. We extend our appreciation to all faculty members and those who directly or indirectly contributed to this project.

~ INDEX ~

Abstract:

Waste management has become one of the major environmental concerns due to the rapid increase in urbanization and industrialization. Manual waste segregation is time-consuming, inefficient, and unhygienic. To address this issue, an AI-Based Automated Waste Segregation System is proposed.

The system uses a conveyor belt mechanism, a camera module, Raspberry Pi controller, and AI-based image classification techniques to identify and segregate different categories of waste such as plastic, metal, wood, paper, and cloth. Images captured by the camera are processed using YOLO and TensorFlow Lite models to detect the waste type in real time.

Based on the classification result, servo motors controlled through the Raspberry Pi direct the waste into corresponding bins automatically. The proposed system minimizes human effort, improves segregation accuracy, enhances recycling efficiency, and provides a scalable and low-cost solution for smart waste management applications.

CHAPTER 1: CONCEPTION OF THE PROJECT

1.1 Introduction:

Improper waste disposal and segregation have become serious environmental challenges across the world. Large quantities of recyclable materials are mixed with general waste, causing pollution and increasing landfill usage. Traditional manual segregation methods are labor-intensive, inconsistent, and expose workers to health hazards.

Artificial Intelligence and Image Processing technologies provide a modern solution to automate waste segregation efficiently. The proposed AI-Based Automated Waste Segregation System combines computer vision, embedded systems, and automation techniques to identify and separate waste materials automatically.

The system uses a camera module to capture waste images and a Raspberry Pi to process them using deep learning algorithms such as YOLO and TensorFlow Lite. Based on the detected waste category, servo motors direct the waste into appropriate bins through a sorting mechanism. The project aims to improve recycling efficiency, reduce human intervention, and promote smart waste management.

1.2 Motivation:

- Increasing environmental pollution due to improper waste management.
- Need for efficient recycling and reusable material separation.
- Reduction of manual labor and human exposure to harmful waste.
- Development of low-cost intelligent automation systems.
- Encouraging smart city and sustainable development initiatives.

1.3 Literature Survey:

Previous studies on waste segregation have used different methodologies to improve classification accuracy and automation. Early systems relied on manual segregation, which was time-consuming, unsafe, and achieved only about 50–60% accuracy. Sensor-based approaches using inductive, capacitive, and moisture sensors improved accuracy to nearly 60–70%, but their material detection capability was limited. Later, image processing techniques such as color, shape, and edge detection achieved around 70–80% accuracy, though they were affected by lighting conditions and overlapping objects. Recent research focuses on deep learning methods like CNN and YOLO, which provide high accuracy of about 90–95% for real-time waste detection. Based on these studies, the proposed system uses YOLO with TensorFlow Lite to achieve efficient and accurate waste segregation suitable for embedded applications.

1.4 Problem Statement:

Improper waste segregation leads to environmental pollution, inefficient recycling, and increased landfill usage. Manual segregation is time-consuming, inconsistent, and unhygienic, especially at the primary collection level. There is a need for a smart, low-cost, and automated system that can accurately classify and segregate waste in real time with minimal human intervention using AI techniques.

1.5 Objectives:

- ✓ To design an AI-based automated waste segregation system.
- ✓ To implement real-time object detection using YOLO.
- ✓ To classify waste materials using image processing techniques.
- ✓ To automate the segregation mechanism using servo motors.
- ✓ To increase the accuracy and speed of waste classification.
- ✓ To reduce manual effort and improve recycling efficiency.
- ✓ To develop a low-cost and efficient smart waste management system.
- ✓ To minimize human exposure to harmful and unhygienic waste.
- ✓ To promote eco-friendly waste disposal and sustainable recycling practices.

1.6 Working Principle:

1. The conveyor belt carries mixed waste materials toward the detection area.
2. A camera module captures images of the waste objects in real time.
3. The captured images are sent to the Raspberry Pi for processing.
4. Image preprocessing techniques such as resizing and normalization are applied.
5. The YOLO/TensorFlow Lite AI model analyzes the image and identifies the waste type.
6. The system classifies waste categories like plastic, metal, paper, wood, or cloth.
7. Based on the detected category, Raspberry Pi sends control signals to the motor driver.
8. The servo motor rotates to direct the waste into the corresponding bin.
9. After segregation, the servo motor returns to its initial position.
10. The conveyor belt resumes movement, and the process repeats automatically for continuous waste segregation.

1.7 Features:

i. AI-based real-time waste classification

ii. Automatic waste sorting mechanism

iii. Custom and modular design

iv. Low-cost embedded implementation

v. Improved segregation accuracy

vi. Scalable architecture for future upgrades

1.8 Areas of Application:

Smart Cities: The proposed system can be widely used in smart city waste management infrastructure for automatic segregation of waste at public collection points. It helps municipalities maintain cleaner surroundings, reduce landfill waste, and improve recycling efficiency. Integration with smart monitoring systems can further support automated waste tracking and management.

Recycling Industries: Recycling industries require proper segregation of waste materials before processing. The AI-based segregation system helps in identifying recyclable materials such as plastic, metal, paper, and wood with better accuracy and speed. This reduces manual labor and increases the efficiency of recycling operations.

Municipal Waste Management Systems: Municipal corporations handle large volumes of mixed waste every day. The proposed system can assist in automatic primary-level segregation at waste collection centers and transfer stations. This improves waste handling efficiency, reduces environmental pollution, and minimizes human exposure to hazardous waste materials.

Educational and Research Applications: The project can be used in engineering colleges, research laboratories, and educational institutions for studying Artificial Intelligence, Embedded Systems, Image Processing, Robotics, and Automation concepts. It serves as a practical model for students and researchers working on smart automation technologies.

Industrial Automation Systems: Industries generating packaging and production waste can use this system for automatic waste sorting and material management. The project supports industrial automation by reducing manual intervention, improving operational efficiency, and enabling systematic disposal of industrial waste materials.

CHAPTER 2: DESIGN OF THE PROJECT

2.1 Design aspect:

The proposed AI-Based Automated Waste Segregation System is designed by integrating mechanical movement, image processing, embedded control, and Artificial Intelligence into a single automated platform. The design mainly focuses on achieving accurate waste identification, real-time processing, and automatic segregation with minimum human intervention. The system consists of a conveyor belt mechanism, camera module, Raspberry Pi controller, AI-based object detection model, motor driver circuit, servo motor sorting mechanism, and waste collection bins.

The conveyor belt continuously transports mixed waste materials toward the detection area. A camera module mounted above the conveyor belt captures images of the waste objects. These images are processed by the Raspberry Pi using image processing and deep learning algorithms. Based on the detected waste category, the Raspberry Pi generates control signals to actuate servo motors through the motor driver circuit. The servo motors rotate to specific positions and direct the waste into corresponding bins.

The complete system is designed with a modular architecture, allowing easy expansion for additional waste categories and future technological upgrades. The proposed model aims to provide reliable, accurate, low-cost, and scalable waste segregation suitable for smart waste management applications.

2.1.1 Hardware Specifications:

(i) Raspberry Pi-

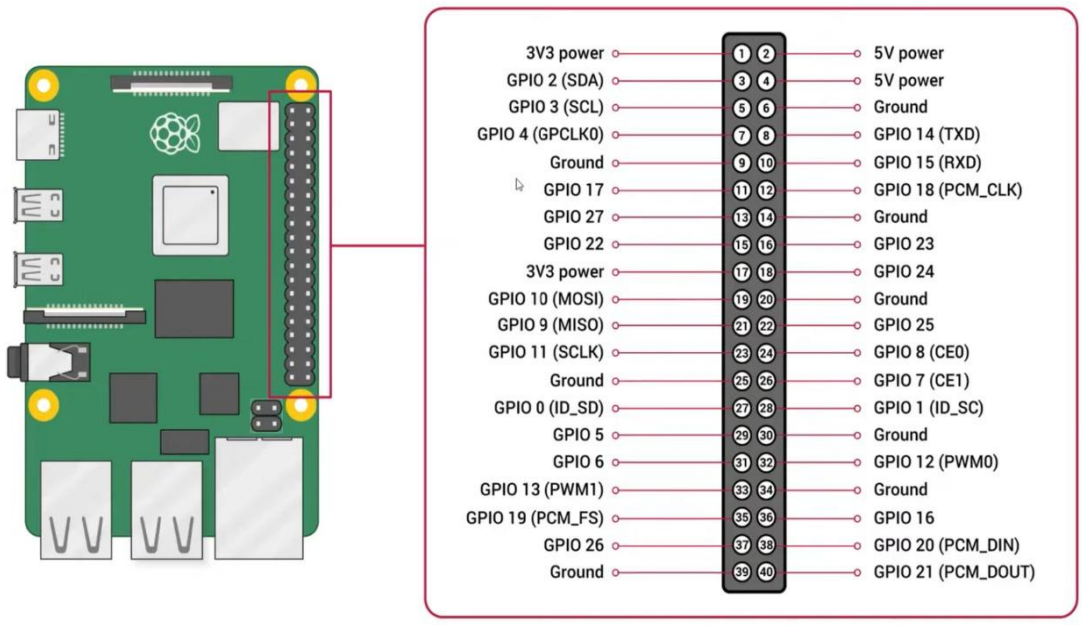


Figure 2.1.a-Raspberry Pi.

• Raspberry Pi 4 Model B

• Processor: 64-bit quad-core ARM Cortex-A72 (ARM v8)

• RAM: 4GB LPDDR4

• 2 micro-HDMI ports (Supports up to 4Kp60)

• 2 USB 3.0 Ports

• 2 USB **2**.0 Ports

• Gigabit ethernet port

• 802.11b/ g/n/ac wireless (Built in Wi-Fi module supporting both 2.4GHz and 5GHz bands.)

• Bluetooth 5.0

• PoE (Power over ethernet, requires PoE HAT)

• Power supply- 5V/3A USB-C

(ii) <u>Servo Motor</u>-

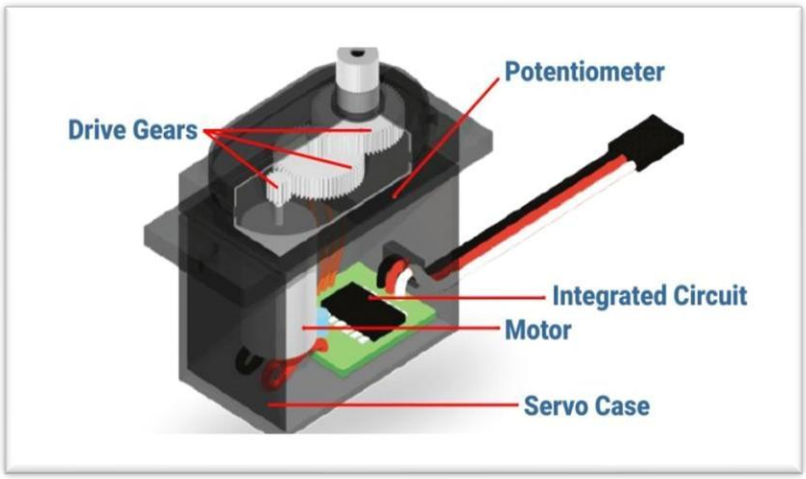


Figure 2.1.b Servo Motor.

- Operating Voltage is +5V typically
- Torque: 2.5kg/cm
- Operating speed is 0.1s/60°
- Gear Type: Plastic
- Rotation : 0°-180°
- Weight of motor : 9gm

(iii) Motor driver BTS7960-

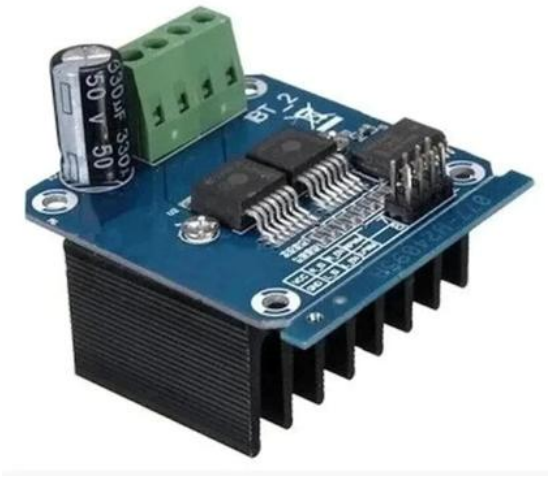


Figure 2.1.c Motor Driver BTS7960.

Features:

- Supports high current motors (up to 43A peak)
- Operating voltage: 5V–27V
- Forward and reverse motor control
- PWM-based speed control
- Low heat generation
- Built-in over-current and over-temperature protection

Working Principle-

- The BTS7960 controls motor speed and direction using PWM signals.
- Forward PWM rotates the motor in one direction, while reverse PWM changes the direction.
- Motor speed is controlled by varying the PWM duty cycle.
- Internal MOSFETs provide efficient switching and smooth motor operation.

(iv) Camera Module-

The camera module is used for capturing real-time images of waste materials moving on the conveyor belt. These images are provided as input to the AI model for classification.

Features of Camera Module:

• High-resolution image capture

• Real-time video streaming capability

• USB interface support

• Compact and lightweight design

• Compatible with Raspberry Pi

Figure 2.1.d WebCam.

Purpose in the Project:

• Captures waste object images

• Provides image input for AI processing

• Supports real-time waste detection

(v) DC Motor-

The DC motor drives the conveyor belt system and ensures continuous movement of waste materials toward the detection area.

Features of DC Motor of RS555:

• Continuous rotational motion

• High torque operation

• Low power consumption

• Simple control mechanism

Figure 2.1.e DC-motor

Purpose in the Project:

• Drives conveyor belt movement

• Controls transportation of waste objects

(vi) Conveyor belt mechanism-

The conveyor belt is a mechanical platform used to transport waste materials continuously through the detection and segregation zones.

Features of Conveyor Belt:

• Continuous waste transportation

• Smooth mechanical movement

• Compact structure

Purpose in the Project:

• Carries waste materials to detection area

• Enables automated segregation process

2.1.2 Software Specifications-

(i) Operating System – Raspberry Pi OS-

Raspberry Pi OS is the primary operating system used in the proposed system. It is a Linux-based operating system specially designed for Raspberry Pi embedded boards. The OS provides a stable, lightweight, and efficient environment for executing Python programs, AI frameworks, image processing libraries, and hardware interfacing modules.

The operating system supports multitasking, GPIO control, USB camera interfacing, file management, and software package installation required for project development. Raspberry Pi OS also provides compatibility with TensorFlow Lite, OpenCV, and other AI-related libraries used in the project.

Features of Raspberry Pi OS:

• Linux-based open-source operating system
• Lightweight and optimized for embedded applications
• Supports Python programming environment
• Compatible with OpenCV and TensorFlow Lite
• Provides GPIO pin access and hardware interfacing
• Supports USB devices and camera modules
• Easy software package installation using terminal commands
• Stable and secure operating platform

Role in the Project:

• Provides software execution environment for the entire system
• Supports AI model deployment and execution

• Enables communication between hardware and software modules
• Handles camera interfacing and image acquisition
• Controls GPIO-based motor operations

(ii) Programming Language – Python-

Python is the primary programming language used for software development in this project. Python is selected because of its simplicity, readability, extensive library support, and strong compatibility with Artificial Intelligence, Machine Learning, and Embedded System applications.

Python allows rapid program development and easy integration with image processing libraries, AI frameworks, and GPIO control modules. It also supports modular programming, making the system easier to debug, modify, and maintain.

Features of Python:

• Simple and easy-to-understand syntax
• Supports Object-Oriented Programming
• Large collection of AI and image processing libraries
• Cross-platform compatibility
• Efficient handling of real-time applications
• Easy integration with Raspberry Pi GPIO interface
• Supports Machine Learning and Deep Learning frameworks

Role in the Project:

• Used for writing the complete system control program
• Handles image capture and preprocessing operations
• Executes AI model inference for waste classification
• Generates motor control signals for servo operation
• Controls conveyor belt synchronization and automation

(iii) OpenCV Library-

OpenCV (Open Source Computer Vision Library) is used for image acquisition and digital image processing operations. It provides various built-in functions required for capturing images from the camera module, preprocessing images, object visualization, resizing, normalization, filtering, and feature enhancement.

The captured waste images are processed using OpenCV before being given as input to the AI model. Proper image preprocessing improves classification accuracy and reduces unwanted noise and disturbances.

Features of OpenCV:

• Open-source computer vision library
• Real-time image and video processing support

• Supports image filtering and enhancement
• Provides image resizing and normalization functions
• Fast execution suitable for embedded systems
• Compatible with Python and Raspberry Pi
• Supports object visualization and image annotation

Role in the Project:

• Captures images from camera module
• Performs image preprocessing operations
• Enhances image quality before AI analysis
• Resizes images according to model input requirements
• Displays detected objects and classification outputs

(iv) TensorFlow-

TensorFlow is an open-source Machine Learning and Deep Learning framework developed by Google AI. It is used in the proposed system for training and executing the YOLO-based waste classification model. TensorFlow provides the necessary libraries and tools for image processing, object detection, and AI model development.

Features of TensorFlow:

• Open-source AI and deep learning framework
• Supports neural network and object detection models
• Compatible with Python and Raspberry Pi
• Supports CPU and GPU processing
• Efficient for image processing applications
• Provides libraries for AI model training and inference

Role in the Project:

• Used for training the waste classification model
• Supports YOLO-based object detection
• Performs image processing and classification
• Enables AI-based waste segregation
• Improves system accuracy and performance

(v) Roboflow-

Roboflow is a cloud-based dataset management and preprocessing platform used during AI model development. It helps in dataset annotation, image labeling, preprocessing, augmentation, and dataset organization required for training the YOLO object detection model.

The platform simplifies dataset preparation and improves model accuracy by generating augmented image variations under different conditions such as brightness, rotation, scaling, and flipping.

Features of Roboflow:

• Dataset annotation and labeling support
• Image preprocessing and resizing tools
• Data augmentation functionality
• Supports YOLO dataset export formats
• Cloud-based dataset management
• Improves training dataset quality
• Simplifies AI model preparation process

Role in the Project:

• Used for labeling waste object images
• Performs image augmentation operations
• Organizes training and validation datasets
• Improves AI model robustness and accuracy
• Generates datasets compatible with YOLO training

(vi) NumPy Library-

NumPy is a numerical computing library used for handling arrays, matrices, and mathematical operations required during image processing and AI inference. Since images are represented as multidimensional arrays, NumPy helps perform efficient matrix manipulation and pixel-level operations.

The library improves computational speed and supports high-performance numerical calculations needed for image preprocessing and AI model execution.

Features of NumPy:

• Fast numerical computation library
• Supports multidimensional arrays and matrices
• Efficient mathematical operation handling
• High-speed image data processing
• Compatible with OpenCV and TensorFlow
• Reduces computational complexity

Role in the Project:

• Handles image array processing
• Performs numerical computations during preprocessing
• Supports matrix operations for AI inference
• Assists OpenCV image manipulation functions
• Improves processing efficiency and execution speed

(vii) RPi.GPIO Library-

The RPi.GPIO library is used for controlling the GPIO (General Purpose Input Output) pins of the Raspberry Pi. This library allows communication between the software program and hardware components such as servo motors, motor drivers, and conveyor belt control systems.

Using this library, the Raspberry Pi generates PWM (Pulse Width Modulation) signals required for servo motor positioning and DC motor control.

Features of RPi.GPIO Library:

• GPIO pin access and configuration support
• PWM signal generation capability
• Supports input and output pin operations
• Simple Python-based control interface
• Real-time hardware communication
• Compatible with Raspberry Pi OS

Role in the Project:

• Controls servo motor rotation angles
• Interfaces Raspberry Pi with motor driver circuit
• Generates PWM signals for motor control
• Controls conveyor belt operation
• Synchronizes hardware actions with AI classification results

2.2 Theoretical Aspects:

The proposed system combines concepts from Artificial Intelligence, Digital Image Processing, Deep Learning, Embedded Systems, and Automation Engineering.

1. Digital Image Processing- Digital Image Processing (DIP) is used to improve the quality of captured waste images before AI analysis. Image processing operations include resizing, normalization, filtering, and enhancement. These operations improve detection accuracy and reduce unwanted noise from images.

2. Artificial Intelligence- Artificial Intelligence enables the system to recognize and classify different waste materials automatically. The AI model learns patterns and visual features from training datasets and uses them to identify waste categories during real-time operation.

3. Convolutional Neural Networks (CNN)- CNNs are deep learning models specifically designed for image recognition applications. CNN layers automatically extract features such as edges, textures, shapes, and object patterns from images.

The CNN model performs:

• Feature extraction
• Pattern recognition
• Object classification
• Confidence score generation

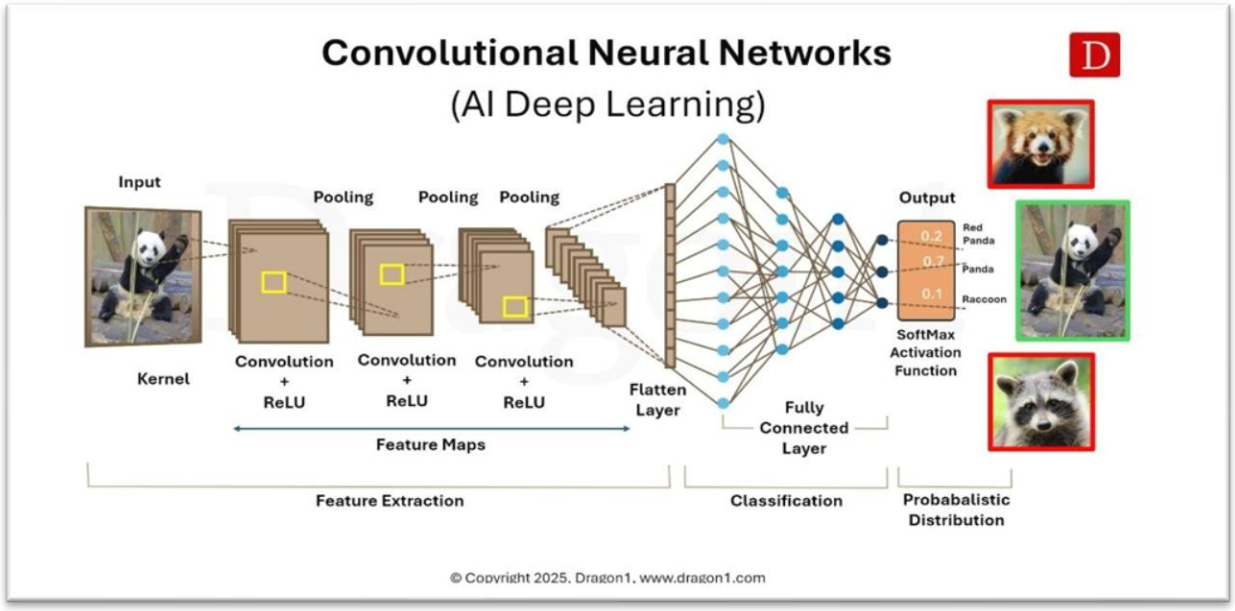


Figure 2.2.a- Convolution Neural Network Technique.

4. YOLO Object Detection- YOLO (You Only Look Once) is a real-time object detection algorithm widely used for fast and accurate detection tasks. Unlike traditional detection methods, YOLO performs object localization and classification in a single step.

Advantages of YOLO include:

• High detection speed
• Real-time operation
• Better accuracy
• Multiple object detection capability
• Efficient embedded system deployment

5. Embedded Systems- Embedded systems combine hardware and software to perform dedicated functions efficiently. Raspberry Pi acts as the embedded controller responsible for system coordination, AI inference, and motor control.

6. Automation and Motor Control- Servo motors and motor drivers automate the waste segregation mechanism. PWM control signals generated by Raspberry Pi determine servo motor angles for directing waste into separate bins.

2.3 IMAGE PROCESSING AND AI MODEL ANALYSIS:

The AI-based waste segregation process begins with image acquisition using the camera module. The captured image is first preprocessed before being provided to the AI model.

Image Preprocessing Steps--

1. **Image Resizing:** The captured image is resized into a fixed resolution suitable for the YOLO model input layer.
2. **Image Normalization:** Pixel values are normalized to improve model accuracy and reduce computational complexity.
3. **Noise Reduction:** Filtering techniques reduce unwanted image noise and improve object visibility.
4. **Feature Extraction:** The CNN model extracts important visual features such as shape, colour, texture, and boundaries.
5. **Classification:** The extracted features are analyzed by the trained YOLO model to identify the waste category, with the help of Softmax activation function.

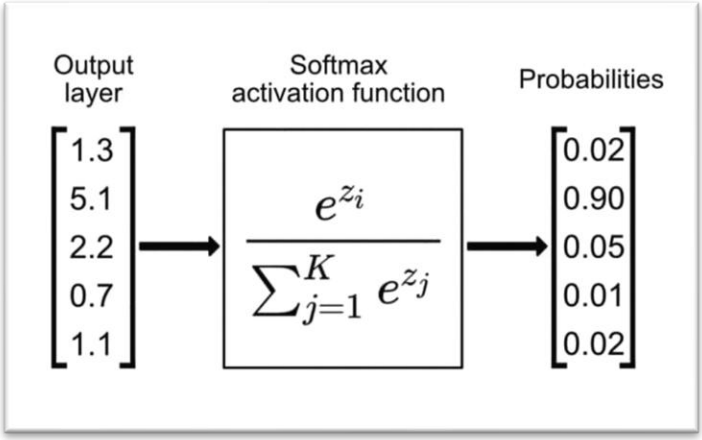


Figure 2.3.a- Showing the operation of softmax function.

The model produces output labels along with confidence scores representing prediction accuracy. The Raspberry Pi selects the highest confidence result and activates the corresponding sorting mechanism.

2.4 DATASET COLLECTION AND TRAINING:

The performance of the AI model mainly depends on the quality of the training dataset. Therefore, proper dataset preparation is an important step in the project.

→**Dataset Collection**: Images of different waste categories such as:

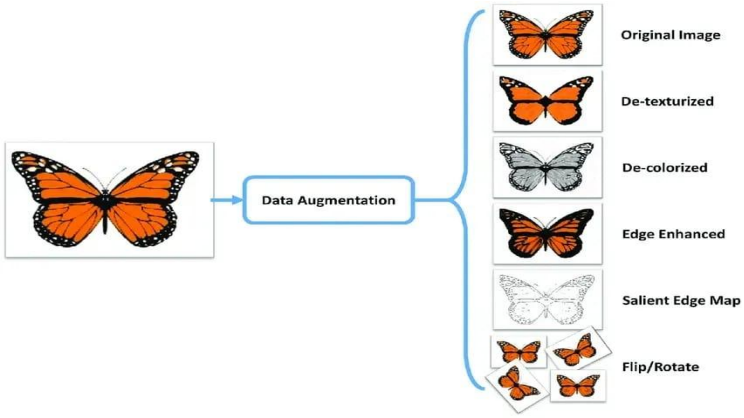
• Plastic
• Metal
• Wood
• Paper
• Others are collected from online sources and manually captured samples.

→**Data Annotation:** Roboflow is used to label waste objects with proper category names. Bounding boxes are created around objects for object detection training.

→**Data Augmentation:** Image augmentation techniques improve model robustness by generating multiple image variations.

Techniques used include:

• Rotation
• Flipping
• Scaling
• Brightness adjustment
• Cropping



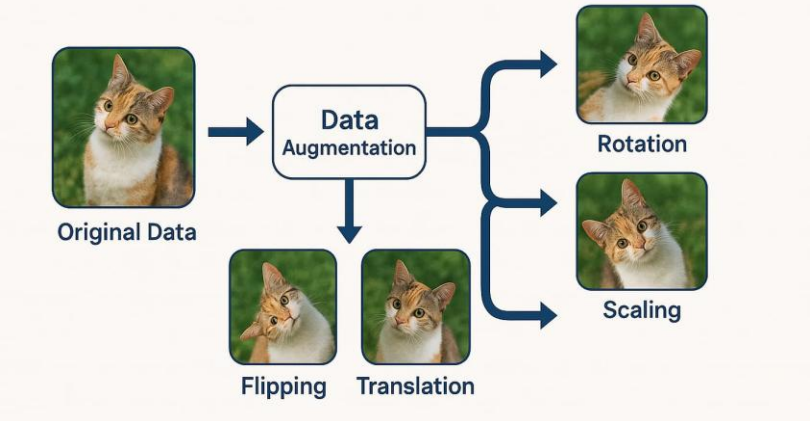


Figure 2.4 a: Samples of data augmentation.

→**Model Training:** The YOLO model is trained using the prepared dataset. During training, the model learns visual patterns associated with different waste materials.

→**TensorFlow Conversion:** After training, the model is converted into TensorFlow format for optimized execution on Raspberry Pi.

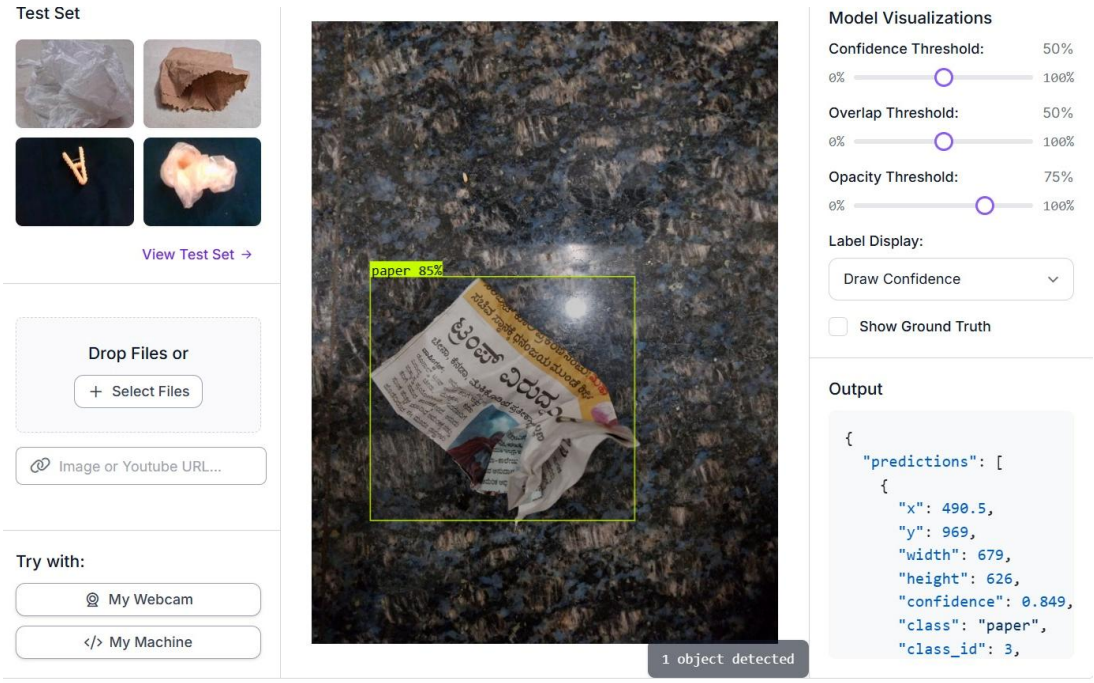
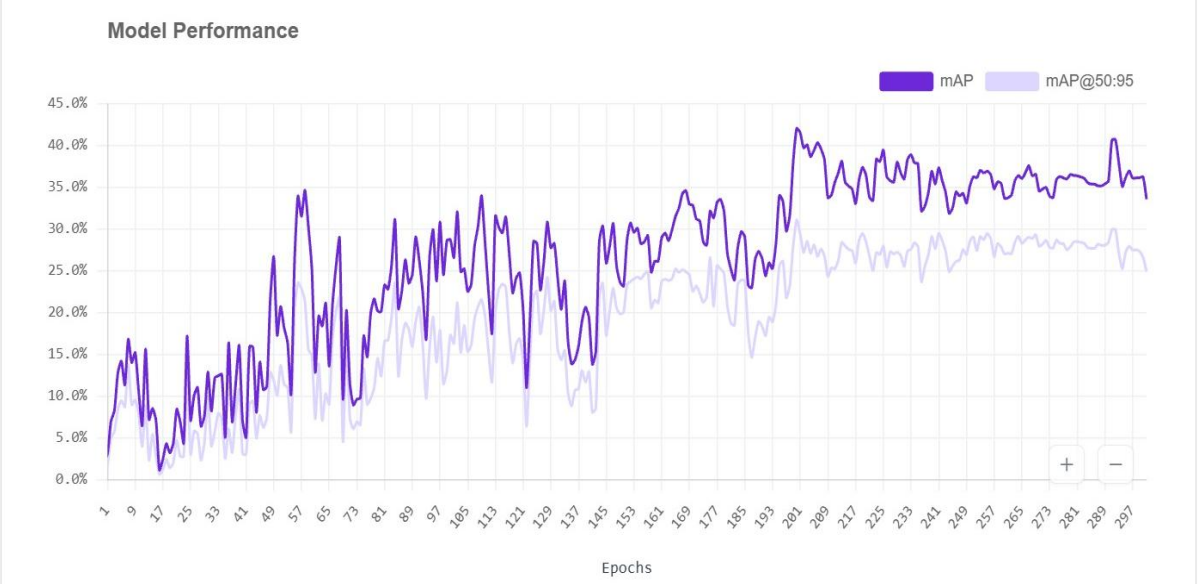


Figure 2.4 a: Screenshot from RoboFlow showing the waste type along with the boundary box and image attributes.



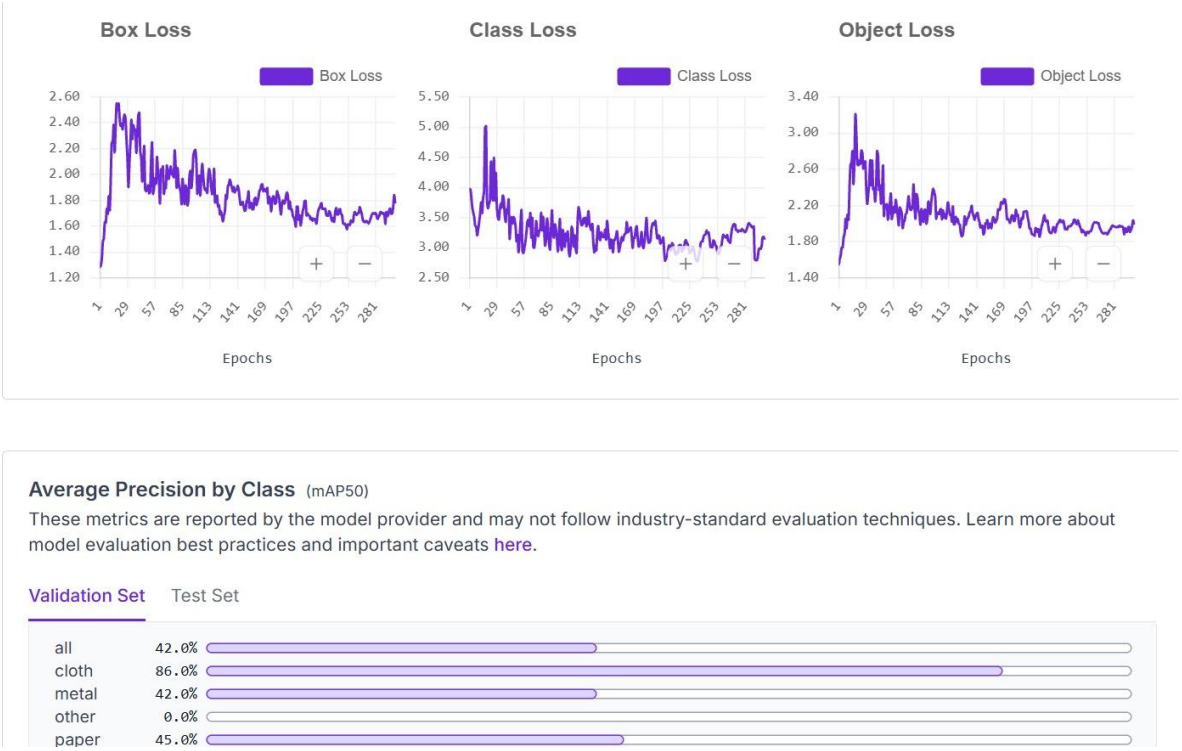


Figure 2.4.b: Model Training Analytics.

The training graphs show that the object detection model gradually improved its learning performance over multiple epochs. The Box Loss, Class Loss, and Object Loss curves initially showed high values and fluctuations because the model started learning object positions, categories, and object presence from scratch. As training progressed, all loss values gradually decreased and stabilized, indicating successful learning, improved object localization, better classification accuracy, and increased confidence in object detection.

The Mean Average Precision (mAP) and mAP@50:95 graphs showed continuous improvement during training, reaching nearly 35%–40% accuracy in the final epochs. This confirms that the model achieved moderate but reliable detection performance. The Average Precision by Class graph revealed that the Cloth category achieved the highest accuracy of about 86%, while Metal and Paper showed moderate performance. However, the Other category achieved very low accuracy due to insufficient training data and unclear object features.

Overall, the model demonstrated effective learning and convergence with good detection capability for certain waste categories. Further improvements can be achieved by increasing dataset size, balancing class samples, applying data augmentation, and fine-tuning model parameters.

2.5 Block Diagram:

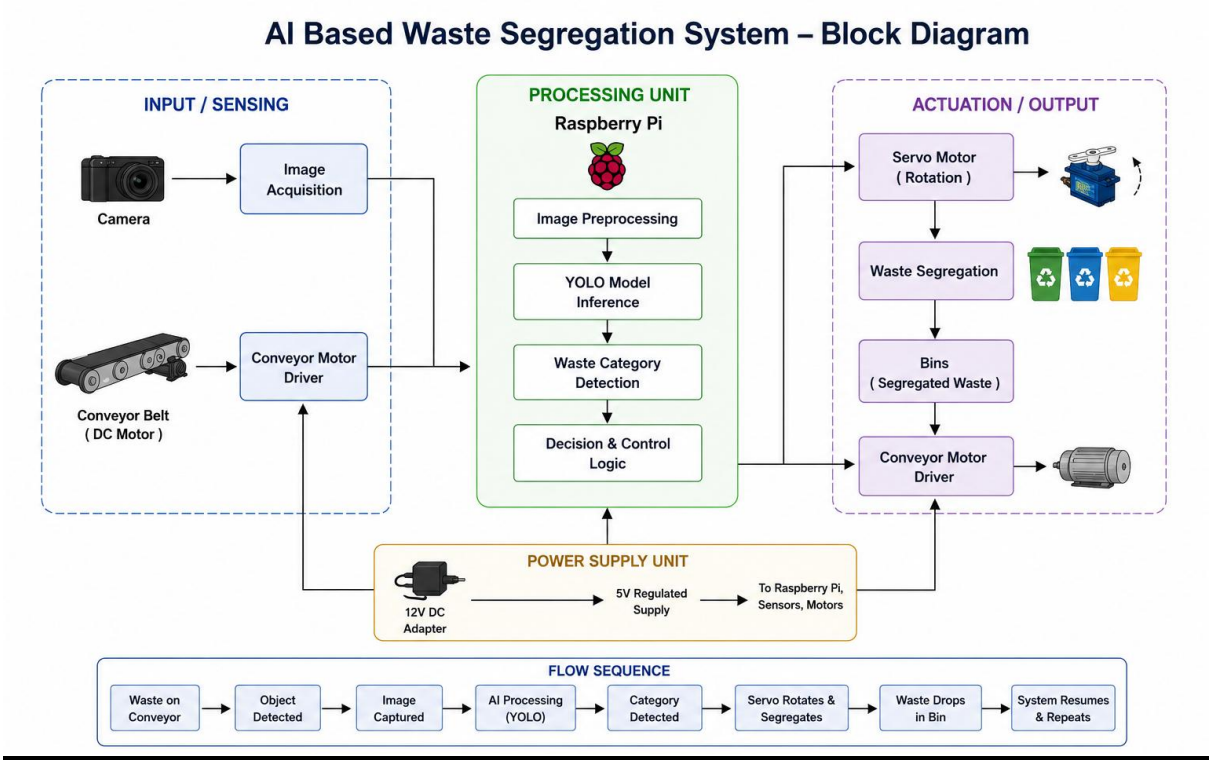


Figure 2.5.Block diagram of the setup.

2.6 Algorithm:

1. Start the system.
2. Initialize Raspberry Pi, camera module, GPIO pins, conveyor motor, and motor driver.
3. Load the trained YOLO object detection model.
4. Start the conveyor belt movement.
5. Continuously monitor for waste object arrival.
6. When an object is detected, stop the conveyor belt temporarily.
7. Capture the image of the waste object using the camera.
8. Preprocess the captured image for AI analysis.
9. Execute YOLO model inference on the image.
10. Identify the type of waste object.
11. Compare detected class with predefined waste categories.

12. Rotate the servo motor to the corresponding bin position.
13. Push or drop the waste into the selected bin.
14. Return the servo motor to its initial position.
15. Restart the conveyor belt.
16. Repeat the process for the next waste object.
17. Stop the system when power is turned OFF.

2.7 Flow Chart:

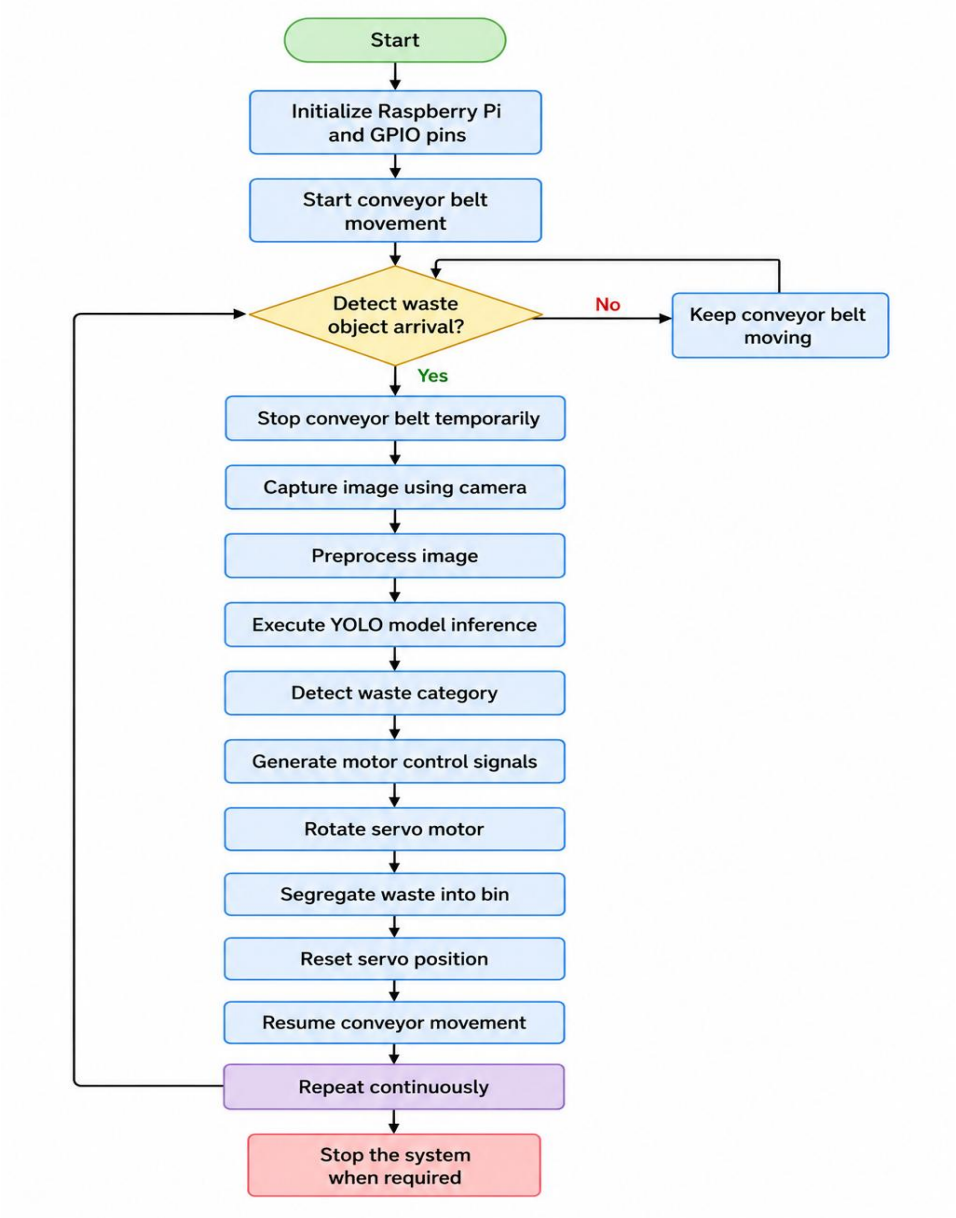


Figure 2.7.a:Flow Chart.

PARAMETER CALCULATIONS

The system parameters are selected based on the system constraints and real-time operation requirements.

Camera Resolution- Image Resolution = 640 × 640 pixels. This resolution provides sufficient image quality while maintaining faster processing speed.

Servo Motor Rotation-Servo Angle Range = 0° to 180° (clockwise and anti-clockwise). Different waste categories are assigned specific angular positions for segregation.

Conveyor Belt Speed- Conveyor speed is adjusted to allow sufficient image processing time before segregation.

AI Model Confidence Threshold- Confidence Threshold = 60% to 80%. Only predictions above the threshold value are considered for segregation to improve accuracy.

Operating Voltages:

- Raspberry Pi = 5V
- Servo Motor = 5V
- DC Motor = 12V

CHAPTER 3: IMPLEMENTATION OF THE PROJECT

3.1 Components Procured:

3.1.1 Hardware Components:

Component	Specification
Raspberry Pi	4 Model B
SD Card	16GB to 32GB
DC motor	12V
Motor Driver	BTS7960
Servo motor	5v
Camera	720P, w200
Battery 18650	12v, 3000mAh
Jumper wire	As per requirement

Table 3.1.a Tabulation of components.

3.1.2 Software Tools:

Tool	Description
Python 3.9.18 (Thony)	Used for writing scripts, Controll the speed of conveyor belt and controlling rotation of servo motor.
Dataset	Collection of labelled waste images(plastic, metal, paper, wood etc.) used for training the model
Roboflow	Used for dataset annotation, preprocessing and augmentation to improve model accuracy
YOLO(CNN model)	Real-time object detection model used to identify and classify waste types.

TensorFlow	Converts and deploys the trained model(TFLite) for efficient execution on embedded system

3.2 Programming Techniques Used:

1. **Real-time video processing (OpenCV):** Continuous frame capture, resizing, and display for live detection.
2. **Image preprocessing:** Normalization and tensor reshaping to prepare input for the deep learning model.
3. **Edge AI inference (TensorFlow):** Efficient on-device model execution using the TFLite interpreter.
4. **Prediction post-processing:** Confidence filtering and class probability analysis to obtain final detection output.
5. **Multi-class classification:** Selection of the highest probability class using `argmax` logic.
6. **Temporal smoothing:** Buffer-based majority voting to ensure stable and noise-free predictions.
7. **State management (FSM logic):** Control flags prevent repeated triggers and manage servo activity flow.
8. **Hardware control (Raspberry Pi GPIO):** PWM-based servo motor control for physical sorting action.
9. **Decision mapping:** Rule-based mapping of detected waste categories to specific servo angles.
10. **System synchronization:** Coordination between camera input, AI inference, and actuator response in real time.
11. **Safe resource handling:** Proper cleanup of camera, GPIO pins, and OpenCV windows after execution.
12. **Event-driven control:** Keyboard input handling for graceful program termination.

3.3 Complete Model Setup:

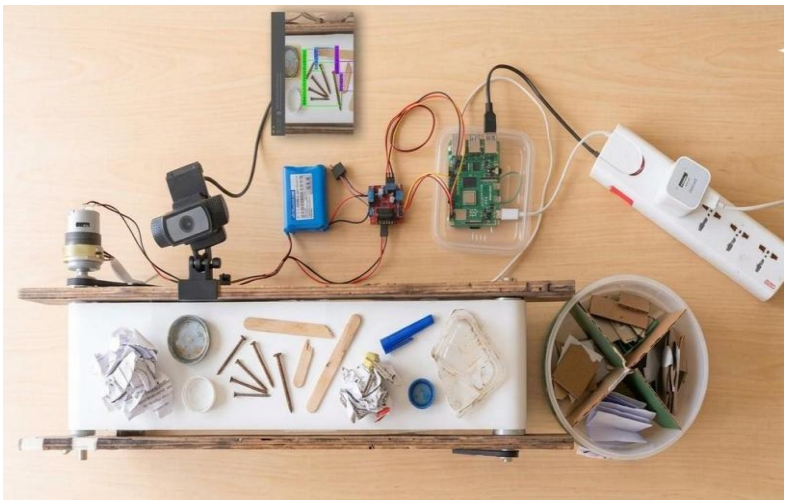


Figure 3.4-Model

3.4 Pseudo-Code:

```
import cv2
import numpy as np
import tensorflow as tf
import RPi.GPIO as GPIO
import time
from collections import Counter

# ====================================
# SERVO SETUP
# ====================================
SERVO_PIN = 17
INITIAL_ANGLE = 90

GPIO.setmode(GPIO.BCM)
GPIO.setup(SERVO_PIN, GPIO.OUT)
pwm = GPIO.PWM(SERVO_PIN, 50)
pwm.start(0)
def set_angle(angle):
```

```
    duty = angle / 18 + 2.5
    GPIO.output(SERVO_PIN, True)
    pwm.ChangeDutyCycle(duty)
    time.sleep(0.7)
    pwm.ChangeDutyCycle(0)
# Initialize servo
set_angle(INITIAL_ANGLE)
print("Servo initialized")

# ====================================
# LOAD MODEL
# ====================================
interpreter = tf.lite.Interpreter(model_path="model.tflite")
interpreter.allocate_tensors()
input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()
labels = ['cloth', 'metal', 'other', 'paper', 'plastic', 'wood']

# ====================================
# CAMERA
# ====================================
cap = cv2.VideoCapture(0)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)
input_size = 640

# ====================================
# DETECTION VARIABLES
# ====================================
prediction_buffer = []
BUFFER_SIZE = 7
```

```
object_locked = False
servo_busy = False
last_detection_time = 0
RESET_DELAY = 2

# ====================================
# SERVO ANGLES
# ====================================
angle_map = {
    "plastic": 30,
    "metal": 60,
    "paper": 120,
    "cloth": 150,
    "wood": 180
}

# ====================================
# MAIN LOOP
# ====================================
while True:
    ret, frame = cap.read()
    if not ret:
        print("Camera Error")
        break
    # ====================================
    # IMAGE PREPROCESSING
    # ====================================
    img = cv2.resize(frame, (input_size, input_size))
    img = img.astype(np.float32) / 255.0
    img = np.expand_dims(img, axis=0)

    # ====================================
```

```
# MODEL INFERENCE
# ====================================
interpreter.set_tensor(input_details[0]['index'], img)
interpreter.invoke()
output = interpreter.get_tensor(output_details[0]['index'])
predictions = output[0]
best_label = None
best_conf = 0

# ====================================
# YOLO DETECTION LOOP
# ====================================
for pred in predictions:
    obj_conf = pred[4]
    # Ignore weak detections
    if obj_conf < 0.25:
        continue
    class_scores = pred[5:]
    class_id = np.argmax(class_scores)
    confidence = float(
        obj_conf * class_scores[class_id])
    if confidence > best_conf:
        best_conf = confidence
        best_label = labels[class_id]

# ====================================
# STORE STABLE PREDICTIONS
# ====================================
if best_label and not object_locked:
    prediction_buffer.append(best_label)
# Keep buffer fixed
if len(prediction_buffer) > BUFFER_SIZE:
```

```
        prediction_buffer.pop(0)

    # ====================================
    # FINAL STABLE DETECTION
    # ====================================
    if (
        len(prediction_buffer) == BUFFER_SIZE
        and not servo_busy
        and not object_locked
    ):
        label_counts = Counter(prediction_buffer)
        stable_label = label_counts.most_common(1)[0][0]
        stable_count = label_counts.most_common(1)[0][1]
        stability_ratio = stable_count / BUFFER_SIZE
        print("Prediction Buffer:", prediction_buffer)
        # Strong stability check
        if stability_ratio >= 0.70:

            print("\nFINAL DETECTION:", stable_label)
            object_locked = True
            servo_busy = True

            # ====================================
            # SERVO ACTION
            # ====================================
            if stable_label in angle_map:
                target_angle = angle_map[stable_label]
                print("Rotating Servo To:", target_angle)
                set_angle(target_angle)
                time.sleep(1.5)
                print("Returning Servo")
                set_angle(INITIAL_ANGLE)
```

```
            last_detection_time = time.time()
            prediction_buffer.clear()
            servo_busy = False

    # ====================================
    # RESET FOR NEXT WASTE
    # ====================================
    current_time = time.time()
    if (
        object_locked
        and best_label is None
        and (current_time - last_detection_time > RESET_DELAY)
      ):
        print("\nReady For Next Waste\n")

        object_locked = False
        prediction_buffer.clear()

    # ====================================
    # DISPLAY
    # ====================================
    display_text = "Waiting for Detection"
    if best_label:
        display_text = (
            f"Detected: {best_label} "
            f"Conf: {best_conf:.2f}"
        )
    cv2.putText(
        frame,
        display_text,
        (20, 40),
        cv2.FONT_HERSHEY_SIMPLEX,
```

```
            0.8,
            (0, 255, 0),
            2
        )
        cv2.putText(
            frame,
            "Press Q or ESC to Exit",
            (20, 80),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.7,
            (0, 0, 255),
            2)
        # ====================================
        # SHOW WINDOW
        # ====================================
        cv2.imshow("AI Waste Segregation", frame)
        key = cv2.waitKey(1)
        if key == ord('q') or key == 27:
            break

# ====================================
# CLEANUP
# ====================================
print("Closing Program")
cap.release()
cv2.destroyAllWindows()
pwm.stop()
GPIO.cleanup()
print("GPIO Cleaned Successfully")
```

CHAPTER 4: RESULTS OF THE PROJECT

4.1 Phase-I Status:

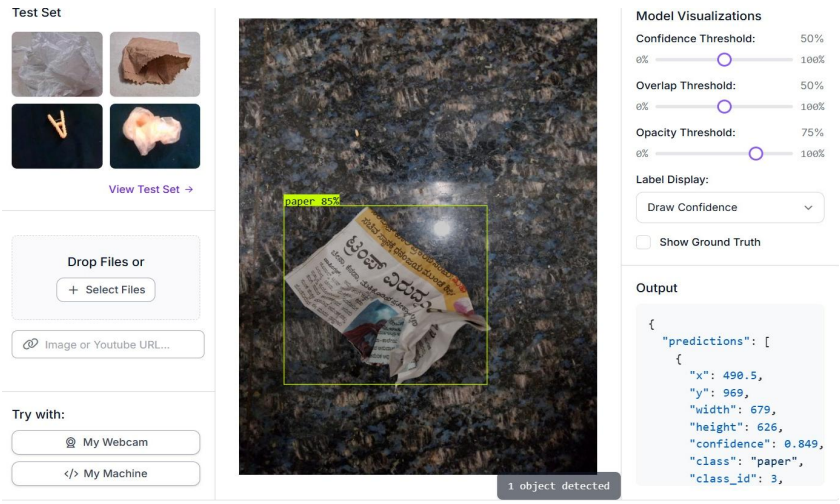
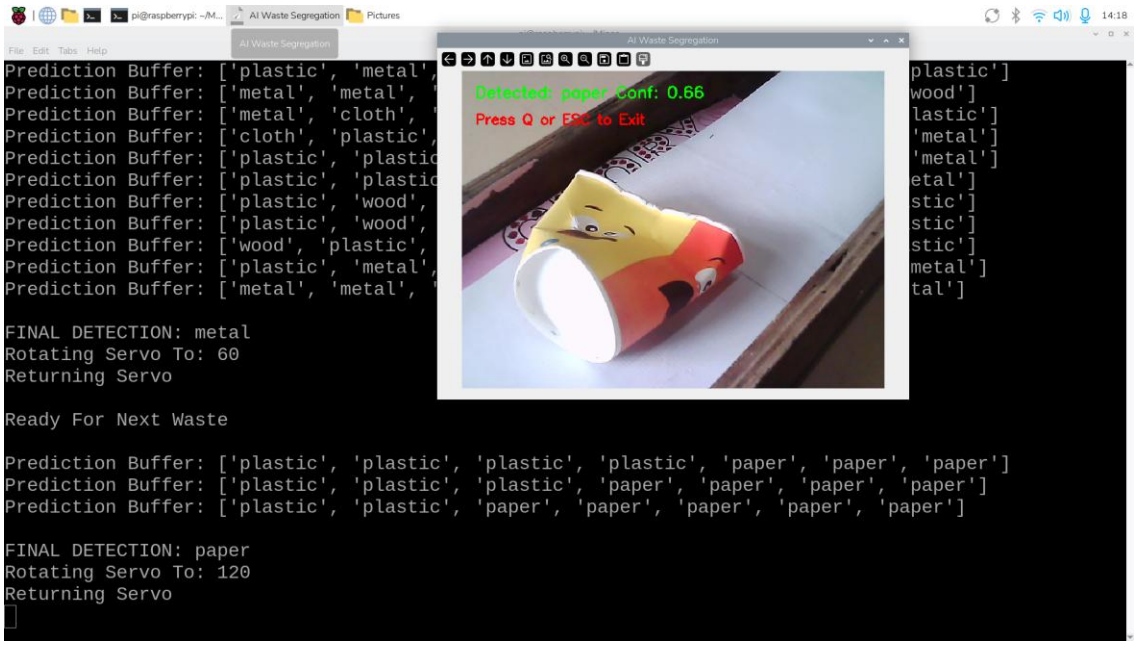


Figure 4.1.a: Dataset Collection and Preparation

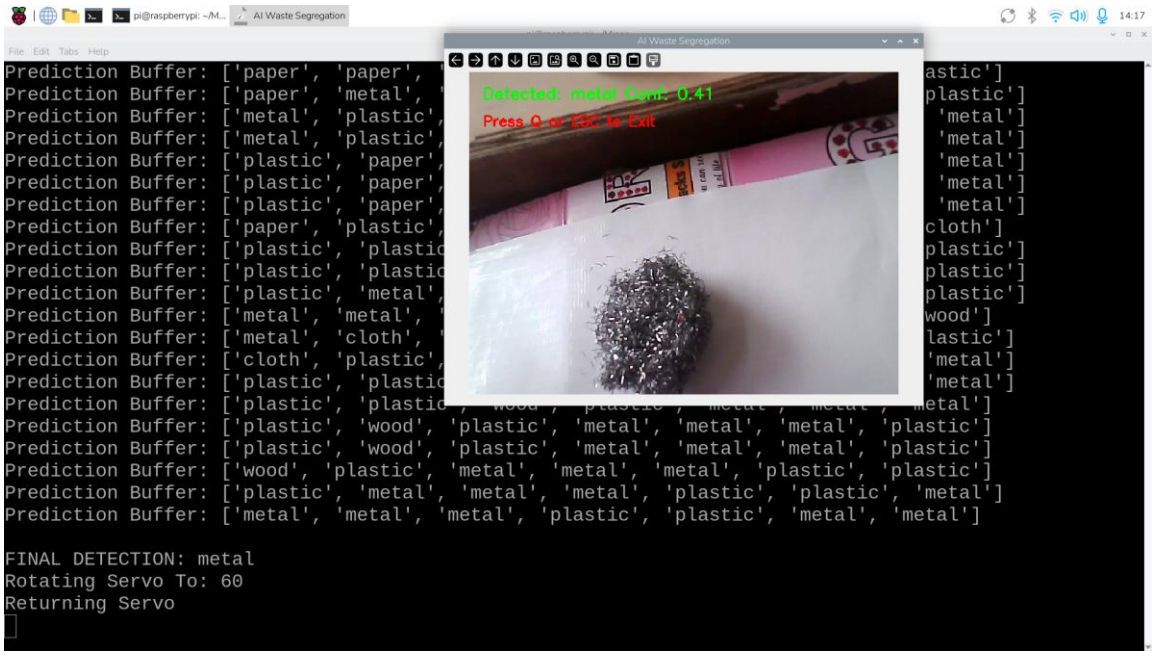
Inference: In Phase-I, a custom dataset of waste images was collected and organized into relevant classes for the segregation system. The dataset was then uploaded to Roboflow for annotation, preprocessing, and augmentation to improve model generalization. Proper labeling and class balancing were ensured to maintain dataset quality, forming a strong foundation for training the waste classification model.

4.2 Phase-II Status:

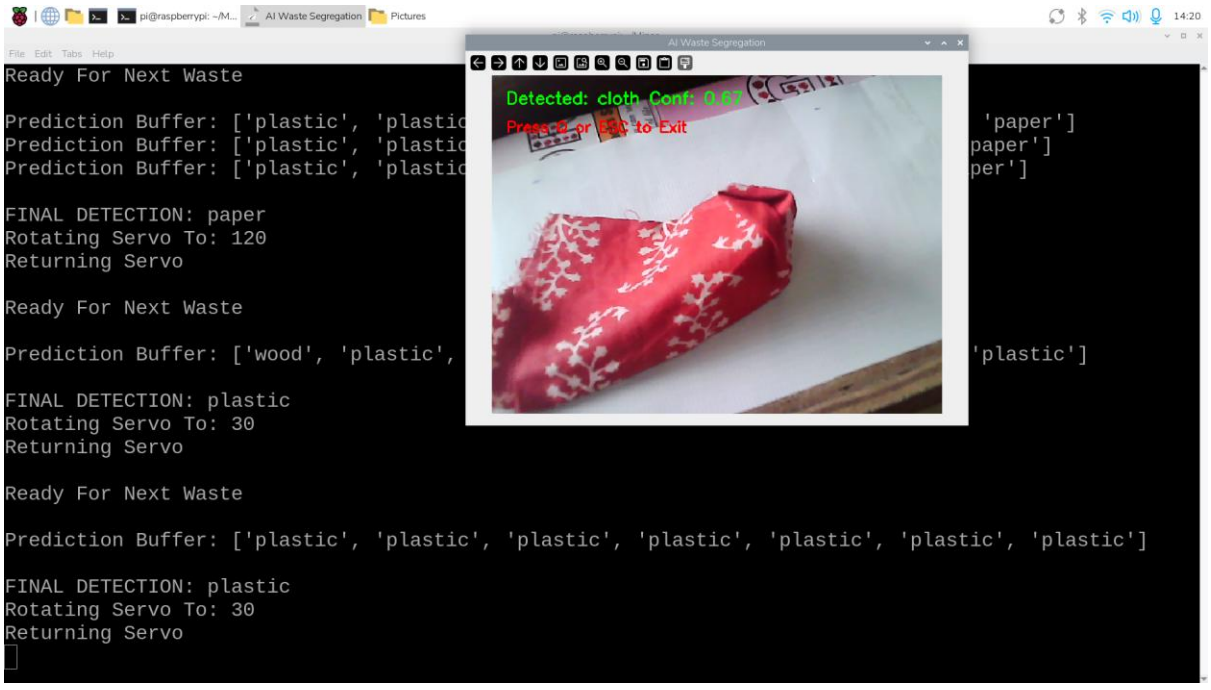
Case-1: Detection of a Paper type-



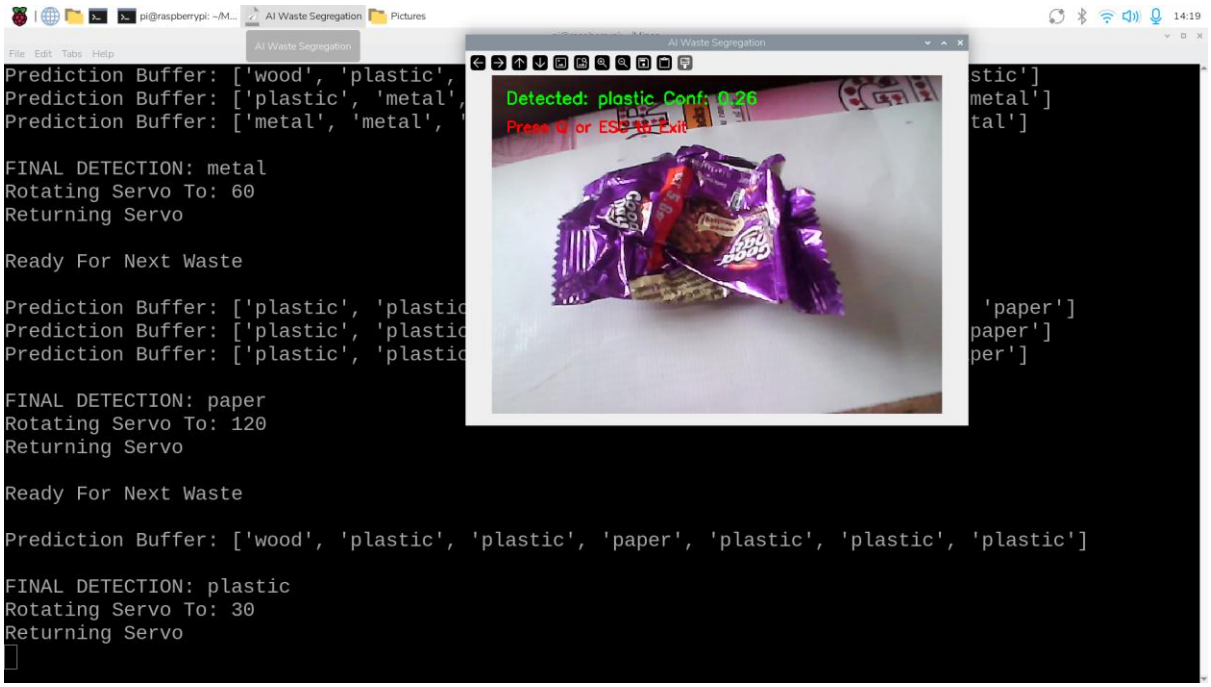
Case-2: Detection of a Metal type-



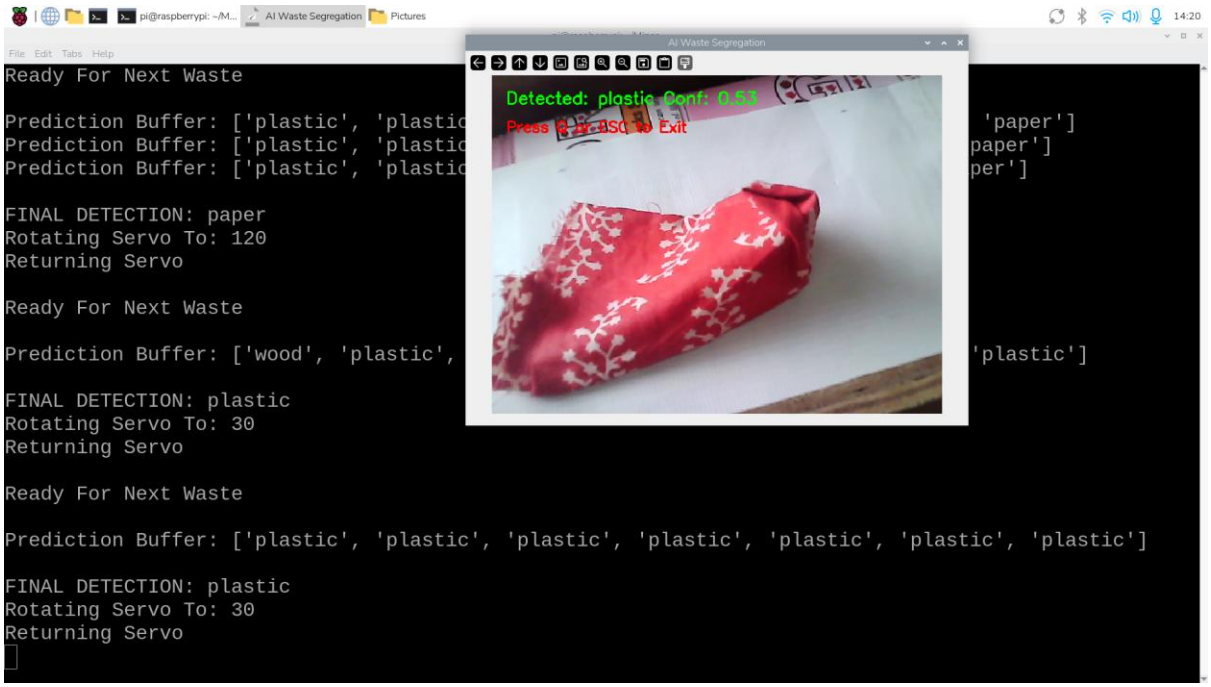
Case-3: Detection of a Cloth type-



Case-4: Detection of a Plastic type-



Case-5: Incorrect detection of a Cloth type as Plastic-



Inference: *Case-1, 2, 3 & 4 are precisly detected objects-*

When the input waste image belongs to a trained class and is clearly visible, the model successfully detects and classifies the object using YOLO-based feature extraction. The prediction is validated through a buffer mechanism, where multiple consecutive frames are analyzed to ensure stability. If at least 70% of the buffered predictions match, the system

confirms a stable and correct classification. Once confirmed, the corresponding label (such as plastic, paper, metal, etc.) is mapped to a predefined servo angle, and the servo motor is triggered to perform accurate segregation. This ensures reliable real-time decision-making and precise waste sorting.

Case-5 incorrect detection-

In cases where the input image is unclear, partially visible, or resembles multiple classes, the model may produce inconsistent or low-confidence predictions. To reduce incorrect classification, a confidence threshold is applied along with object confidence filtering. Additionally, a buffer-based majority voting mechanism is used to stabilize predictions across multiple frames. If the stability ratio does not reach 70%, the detection is not considered valid, and no servo action is triggered. This prevents false activations and ensures that uncertain predictions do not lead to incorrect waste segregation.

4.3 Challenges Faced During Project Development:

- **Raspberry Pi setup and dependency issues (cv2, RPi.GPIO, TensorFlow):**

  -The system encountered import errors due to mismatched Python versions and missing libraries.

  -This was resolved by configuring a proper Python 3.9 virtual environment and reinstalling all required dependencies correctly.

- **Real-time processing delay and camera lag:**

  -The live detection system showed reduced performance and frame delays on Raspberry Pi.

  -This was improved by optimizing frame resolution and streamlining the inference pipeline for faster processing.

- **TensorFlow Lite model integration and label inconsistencies:**

  -Incorrect predictions occurred due to improper label mapping and preprocessing mismatch.

  -This was corrected by standardizing the label file and ensuring consistent input preprocessing before inference.

- **Servo motor synchronization with detection output:**

  -The servo actuation was not consistently aligned with classification results.

  -This was addressed by implementing GPIO-based event triggering with controlled timing delays.

- **Conveyor belt control using BTS7960 motor driver:**

  -Motor response was unstable due to inconsistent GPIO signal handling. This was resolved by refining the control logic and ensuring proper switching sequence for stable operation.

4.5 Limitations:

1. **Dependence on Lighting Conditions-** The accuracy of image detection may reduce in poor lighting or shadow conditions. Improper illumination can affect camera image quality and object classification.

2. **Limited Dataset Accuracy-** The AI model trained using YOLO depends on the quality and quantity of training images. Unknown or complex waste objects may not be classified correctly.

3. **Processing Speed Constraints-** Since the system uses Raspberry Pi, high-resolution image processing and AI inference may take additional time compared to high-performance computers.

4. **Difficulty in Detecting Mixed Waste-** Waste items containing multiple materials (such as plastic-coated paper or food-covered plastic) may lead to incorrect segregation.

5. **Mechanical Wear and Maintenance-** Conveyor belts, motors, and moving mechanisms require periodic maintenance due to continuous operation.

4.6 Future Scope:

1. Improved AI Accuracy- Future systems can use larger datasets and advanced deep learning models to improve classification accuracy for different types of waste.

2. Cloud-Based Monitoring- The project can be connected to IoT platforms for real-time monitoring, data storage, and remote control through the internet.

3. Automatic Waste Compression- Additional mechanisms can be added to compress segregated waste automatically for better storage and transportation.

4. Multi-Category Waste Segregation- The system can be extended to separate more categories such as metal, glass, paper, organic waste, and e-waste individually.

5. Integration with Smart Cities- The project can be integrated into smart city infrastructure for automated municipal waste management systems.

6. Industrial Scale Implementation- The proposed prototype can be upgraded into a large-scale industrial waste segregation system for factories and recycling plants.

➢ Conclusion:

The proposed AI-based waste segregation system provides an efficient and intelligent solution for automatic waste classification and management. By combining Raspberry Pi, camera-based image processing, conveyor mechanisms, and the YOLO algorithm, the system is capable of identifying and separating different types of waste with reduced human intervention.

The project helps improve waste management efficiency, reduces manual sorting effort, and supports environmentally friendly recycling processes. The use of Artificial Intelligence and automation increases the speed and accuracy of waste segregation compared to traditional methods.

This system can play an important role in smart waste management applications such as smart cities, recycling industries, educational institutions, and municipal waste collection systems. Although the prototype has certain limitations, it provides a strong foundation for future advancements in intelligent and automated waste management technologies.

Overall, the project demonstrates how AI and embedded systems can be effectively used to solve real-world environmental problems and contribute towards a cleaner and more sustainable future.

➢ References:

[1]. Textbooks: Simon Monk –"Programming the Raspberry pi – Getting started with Python", 2nd Edition, McGraw-Hill.

[2]. Custom Dataset:https://demo.roboflow.com/my-first-project-f1bcf/1?publishable_key=rf_JWRSdj9ynlges3WIDtJHWFsWUHH2

[3]. Motor Driver BTS7960 Datasheet: https://38-3d.co.uk/blogs/blog/using-the-bts7960-with-the-raspberry-pi?srsltid=AfmBOooVLcYQzxXSr-IkImFBjEmdR4rt6BeeJnSJCZuWzdepyMnE7q-y

[4]. Python 3.9.18 version (Downgrading verison): https://www.python.org/ftp/python/3.9.18/Python-3.9.18.tgz

[5]. G. Thung and M. Yang, "TrashNet: A Deep Learning Dataset for Garbage Classification," Stanford University, 2016.

[6]. S. Mittal, R. Sharma, and A. Kumar, "Waste Classification Using Deep Learning Convolutional Neural Networks," International Journal of Computer Applications, vol. 182, no. 38, pp. 1–6, 2019.

[7]. A. Gupta and P. Jain, "Smart Waste Management System Using Machine Learning and IoT," IEEE Access, vol. 8, pp. 124–135, 2020.

APPENDIX-I

Downgrading Python in Raspberry Pi

(From v3.13.5 → v3.9.18)

Step 1: Check Current Python Version-

Open terminal: `python3 --version`

Example output:Python 3.13.0

Step 2: Install Required Dependencies-

Update packages first: `sudo apt update`

`sudo apt upgrade`

Now install required build tools and libraries:

```
sudo apt install build-essential libssl-dev zlib1g-dev \
libncurses5-dev libffi-dev libsqlite3-dev \
libreadline-dev libbz2-dev
```

Step 3: Download Python 3.9.18-

Move to Downloads folder: `cd ~/Downloads`

Download Python source code:

```
wget https://www.python.org/ftp/python/3.9.18/Python-3.9.18.tgz
```

Extract the file: `tar -xvf Python-3.9.18.tgz`

Go inside extracted folder: `cd Python-3.9.18`

Step 4: Configure Installation-

Run: `./configure --enable-optimizations`

This prepares Python for compilation.

Step 5: Compile Python-

`make -j4`

-j4 uses 4 CPU cores for faster compilation.

(Compilation may take 20–40 minutes on Raspberry Pi.)

Step 6: Install Python 3.9.18-

Use: `sudo make altinstall`

Do NOT use make install, altinstall keeps system Python safe avoids breaking RPi OS

Step 7: Verify Installation-

Check installed version: `python3.9 --version`

Expected: Python 3.9.18

Step 8: Create Virtual Environment-

Install venv package: `sudo apt install python3.9-venv`

Create virtual environment: `python3.9 -m venv myenv`

Activate it: **source myenv/bin/activate**

Now terminal shows: (myenv)

Step 9: Install Required Packages Inside Environment-

Upgrade pip: `pip install --upgrade pip`

Install packages: `pip install opencv-python numpy tensorflow`

Step 10: Deactivate Environment-

When finished: **deactivate**

~ ~ ~ X ~ ~ ~